Informe de Implementación Arquitectura Backend: Proyecto SportMind

1. Resumen Ejecutivo

Este documento detalla la arquitectura, configuración y despliegue del ecosistema backend en Amazon Web Services (AWS) para el proyecto SportMind. El objetivo principal fue establecer una canalización segura, escalable y automatizada para capturar métricas psicomotrices desde dos orígenes distintos (el panel de control web estático y el visor Meta Quest de Realidad Virtual) y centralizarlas en un bucket de almacenamiento S3.

2. Configuración de Almacenamiento (Amazon S3)

El bucket S3 actúa como el repositorio centralizado ("Single Source of Truth") para todas las sesiones de los atletas.

- **Estructura de Directorios:** Se definió el prefijo `JSON_SM/` como el directorio raíz donde aterrizan todos los archivos de métricas.
-
- **Indexación Dinámica:** Para evitar tener las URL codificadas (hardcodeadas) en el frontend, se habilitaron los permisos `s3:ListBucket` y la configuración CORS (Cross-Origin Resource Sharing) en el bucket. Esto permite que el dashboard web solicite un archivo XML con el índice completo de los objetos y los procese mediante `DOMParser`.
-
- **Convención de Nombres:** Los archivos generados adoptan el formato `sportmind_data_[TIMESTAMP]_[UUID].json` para garantizar unicidad y trazabilidad.
-

3. Gestión de Puntos de Enlace (Amazon API Gateway)

Se implementó una API HTTP para exponer el backend a internet. Se habilitó la **Implementación automática** en la etapa `prod`, garantizando que cualquier cambio en las rutas se publique en tiempo real sin intervención manual.

Se diseñaron dos rutas independientes para manejar los distintos flujos de autenticación de los clientes:

Ruta de API Gateway	Cliente Destino	Tipo de Autorización	Mecanismo de Seguridad
`POST /metricas`	Dashboard Web (Astro)	**Cognito JWT**	Validación de Token JWT inyectado desde el `localStorage` del navegador web.
`POST /vr-metricas`	Visor VR (Meta Quest)	**Ninguna (Abierta)**	Se delega la validación de seguridad a nivel de código (Lambda) usando un Custom Header.

4. Lógica de Procesamiento (AWS Lambda)

Se desarrolló y desplegó una función Serverless en Python (`SportMind-GuardarMetricas`) encargada de actuar como puente entre API Gateway y S3.

Características principales del código:

- **Enrutador Unificado:** La función intercepta el objeto `event` de AWS y enruta las peticiones evaluando `event.get('path')` y `event.get('rawPath')`. Soporta activamente las variaciones de ruta generadas por la etapa de API Gateway (ej. `/metricas`, `/vr-metricas`, `/prod/metricas`, `/prod/vr-metricas`).
-
- **Validación de Seguridad Custom (VR):** Para la ruta proveniente de Unity, el código extrae los encabezados (transformados a minúsculas para evitar errores de formato) y verifica el valor de `x-api-key` o `authorization` contra la contraseña secreta definida (`SportMind-VR-2026`). Si no coincide, rechaza la petición con un error `403 Forbidden`.
-
- **Generación y Almacenamiento:** Valida que el cuerpo sea un JSON estructurado, inyecta metadatos adicionales (como el `session_id` generado vía `uuid4` y el timestamp), y ejecuta un `put_object` hacia S3 mediante la librería `boto3`.

5. Integración de Clientes y Refactorización

Ecosistema Web (Astro + Tailwind)

- Se refactorizó el servicio sportmindService.ts. El componente ahora realiza una petición GET a la raíz del bucket con el parámetro ?prefix=JSON_SM/, procesa el XML devuelto y lanza promesas en paralelo (usando Promise.all) para recuperar y unificar todos los archivos .json en el dashboard al instante.
-

Ecosistema Realidad Virtual (Unity / C#)

- Se ajustó la clase APIManager.cs. Se eliminó la concatenación rígida del sufijo /data en la variable constructora (ApiDataUrl => apiBaseUrl), permitiendo que las peticiones se dirijan exactamente a la ruta designada en AWS.
-
- La URL de invocación de producción se asignó a la variable apiBaseUrl.
-
- La clave de acceso (SportMind-VR-2026) se configuró en el inspector de Unity, la cual el script inyecta dinámicamente en el encabezado x-api-key de la petición UnityWebRequest.
-

6. Pruebas y Resolución de Problemas (Troubleshooting)

Durante el despliegue final se aislaron y resolvieron los siguientes cuellos de botella:

- **Restricciones de IAM:** Se detectó un bloqueo institucional (explicit deny en la política voc-cancel-cred) que impedía el uso de Planes de Uso nativos de AWS. Esto impulsó la decisión arquitectónica de trasladar la validación de la API Key directamente al código Python de la Lambda.
-
- **Pruebas de Red (CORS):** Al realizar el Quality Assurance final utilizando herramientas de testeo HTTP (Hoppscotch), se aplicó un Proxy intermediario para eludir las políticas estrictas de CORS del navegador durante la simulación de los POST.
-
- **Mapeo de Rutas (Formato HTTP API):** Se identificó y corrigió una discrepancia en cómo API Gateway v2 (HTTP API) entrega la ruta a Lambda frente al modelo antiguo (REST API). Se ajustó el script Python para capturar correctamente la cadena completa incluyendo el nombre de la etapa (/prod).